Speaker Notes

Lifecycle Portfolio Allocation via Reinforcement Learning
CME 241 – Winter 2026

Patrick Flanagan Jack Zhang Jeffrey Xue

Contents

1 Slide 1 – Title **3**

2 Slide 2 – Outline **3**

3 Slide 3 – The Lifecycle Allocation Problem **3**

4 Slide 4 – MDP Formulation **3**

5 Slide 5 – Transition Dynamics **3**

6 Slide 6 – Phase 2: Exact DP Solution **4**

7 Slide 7 – The Curse of Dimensionality **4**

8 Slide 8 – Phase 3: RL Architecture **4**

9 Slide 9 – Linear Q-Learning **5**

10 Slide 10 – SARSA: On-Policy TD Control **5**

11 Slide 11 – DQN: Deep Q-Network **6**

12 Slide 12 – PPO: Proximal Policy Optimization **6**

13 Slide 13 – Training Convergence **7**

14 Slide 14 – Monte Carlo Evaluation **7**

15 Slide 15 – Learned Policies: What Do the Agents Do? **7**

16 Slide 16 – Wealth Trajectories and Terminal Wealth **8**

17 Slide 17 – Comparing All Four RL Agents **8**

18 Slide 18 – Conclusions **8**

19 Slide 19 – Thank You & Q&A **9**

1 Slide 1 – Title

[Patrick speaks – ∼15 sec]

"Hi everyone – we're Patrick, Jack, and Jeffrey. Our project applies reinforcement learning to the lifecycle portfolio allocation problem. The question we're asking is: how should a person invest and consume over a 40-year working life to maximize their lifetime utility? We'll walk through the problem formulation, our exact dynamic programming solution, why DP breaks down at scale, and how four different RL algorithms handle the harder version of the problem."

2 Slide 2 – Outline

[Jack speaks – ∼20 sec]

"Here's our roadmap for the next fifteen minutes. We start with the problem formulation as a Markov Decision Process, cover how we solved the small version exactly with backward-induction DP, show why that exact approach fails when you add realism, then spend most of our time on the four RL algorithms and what we found."

3 Slide 3 – The Lifecycle Allocation Problem

[Jack speaks – ∼1.5 min]

"Let's ground the problem economically. Imagine you are 30 years old. Every year for the next 40 years you receive labor income – but that income fluctuates: maybe you get promoted, maybe you lose your job. At each year you face two coupled decisions: how much of your current wealth to *consume* versus *save*, and how to split your savings across assets with different risk-return profiles.

This is the central problem of personal finance. Every target-date fund you can buy at Vanguard or Fidelity is a proposed answer to this problem. The industry's standard answer is a fixed glidepath: start with 80% stocks at age 30, gradually shift toward bonds as you approach retirement. That rule ignores your actual wealth level, your current income, and what the market is doing right now.

We're asking: can an RL agent, optimizing from scratch using only simulated experience, find a better policy? And specifically, can it discover the economic intuition that theoretical finance – Merton's continuous-time model – tells us the optimal policy should have, namely state-dependent allocation and smooth consumption?"

4 Slide 4 – MDP Formulation

[Jack speaks – ∼2 min]

"We formulate this as a finite-horizon Markov Decision Process. Let's be precise about each component.

The **state** at time t is the tuple (t, W_t, y_t, z_t): the current period, current wealth W_t, income level y_t, and market regime z_t. This is Markovian – everything the agent needs to make an optimal decision is captured here.

The **action** is a pair $(c, \boldsymbol{\pi})$: the fraction $c \in [0, 1]$ of wealth consumed today, and the portfolio weights $\boldsymbol{\pi} = (\pi_{\text{stocks}}, \pi_{\text{bonds}}, \pi_{\text{cash}})$ satisfying $\pi_i \geq 0$ and $\sum_i \pi_i = 1$.

The **reward** is discounted CRRA utility of consumption:

$$R_t = \beta^t \cdot u(C_t), \quad u(c) = \frac{c^{1-\gamma}}{1-\gamma}, \quad \gamma = 2, \quad \beta = 0.96.$$

With $\gamma = 2$, this simplifies to $u(c) = -1/c$. This utility function is strictly concave – the first dollar consumed is worth far more than the thousandth. The agent is inherently risk-averse: a 10% drop in consumption hurts more than a 10% gain helps. This is critical for understanding why RL agents prefer smooth consumption over volatile terminal wealth.

The **objective** is to maximize expected total discounted utility:

$$\max_{\pi} \mathbb{E}\left[\sum_{t=0}^{T-1} \beta^t u(C_t) + \beta^T u(W_T)\right].$$

The terminal term $\beta^T u(W_T)$ gives utility from the retirement nest egg – the agent is rewarded not just for consuming along the way, but for arriving at retirement with some wealth.

One subtle but important implementation point: because β^t is already baked into each reward R_t when we compute it, we set the RL discount factor $\gamma_{\text{RL}} = 1$ in all our agents. Setting $\gamma_{\text{RL}} = 0.96$ would double-discount. This is a common implementation mistake in lifecycle models."

5 Slide 5 – Transition Dynamics

[Jeffrey speaks – ∼1.5 min]

"Here's the stochastic structure that makes this problem hard. The wealth evolution equation is:

$$W_{t+1} = (W_t - C_t) \cdot R_t^{\boldsymbol{\pi}} + y_{t+1},$$

where $R_t^{\boldsymbol{\pi}} = \boldsymbol{\pi} \cdot \mathbf{r}_t$ is the gross portfolio return. Income arrives at the start of next period.

Both y_t and z_t evolve as Markov chains. Income has three states – low (\$3), medium (\$10), high (\$25) – with a persistent transition matrix: 70% probability of staying in your current income level. Regime also has three states: bear, normal, bull – again persistent, with 50–60% probability of staying in the current regime.

Asset returns are *regime-dependent*. In a bear market, stocks have 40% probability of returning −18% and only a 20% chance of a positive return. In a bull market, stocks have a 40% chance of returning +25%. Bonds and cash are far more stable – bonds return 1–5% regardless of regime, cash returns 0.5%.

This regime-dependence is economically important. A rational investor should hold more equities in a bull regime and rotate toward bonds in a bear regime. Can RL agents learn this? Yes – and we'll see it in the policy heatmaps.

The key challenge is that all three sources of randomness – returns, income, regime – are correlated in time through their Markov structures. The agent has to learn a policy robust to all of these simultaneously, over a 40-year horizon."

6 Slide 6 – Phase 2: Exact DP Solution

[Jack speaks – ∼2 min]

"Before we go to RL, we built an exact solution using backward induction on a simplified version of the problem. This gives us a theoretical benchmark and, crucially, tells us what the optimal policy actually looks like – so we can check whether our RL agents rediscover it.

The Phase 2 setup: 25-period horizon, 2 income states, 2 market regimes, 3 assets. Wealth is discretized to a 121-point grid. This gives 121 wealth levels $\times$ 2 income states $\times$ 2 regimes = 484 states per period.

The action space has 11 consumption fractions and 15 discrete allocations, giving 165 actions.

The Bellman equation for the value function is:

$$V_t(s) = \max_{a \in \mathcal{A}} \left[R(s,a) + \mathbb{E}_{s' \sim P(\cdot|s,a)} \left[V_{t+1}(s') \right] \right].$$

At $t = T$, the boundary condition is $V_T(s) = \beta^T u(W)$. We work backward: at each t, for each of the 484 states, we evaluate all 165 actions and take the argmax. Total computations: $25 \times 484 \times 165 \approx 2$ million – done in under half a second in Python.

The optimal policy from DP is what classical finance theory predicts: young investors take maximum equity exposure because they have time to recover from crashes, and they gradually shift toward bonds as retirement approaches. Consumption rises over time as wealth grows. This is the Merton glidepath.

Compared to three static baselines (60/40 balanced, 100% stocks, 4% withdrawal), the DP policy outperforms all of them on both expected utility and risk-adjusted terminal wealth.

But this only works because we kept the problem artificially small. Watch what happens when we add realism."

7 Slide 7 – The Curse of Dimensionality

[Jeffrey speaks – ∼1.5 min]

"Here's why DP fails and why we need RL. Phase 2 had a manageable state space. Now in Phase 3, we make the problem more realistic: three income levels instead of two, three market regimes instead of two, and a 40-year horizon instead of 25.

If we tried to use the same DP approach with the Phase 3 setup – even with the same discretized wealth grid – the state space grows by roughly $3/2 \times 3/2 \times 40/25 \approx 3.6\times$. But more fundamentally, if we wanted *continuous* wealth (as the real problem demands), the table would be infinite.

This is the curse of dimensionality: the state space grows exponentially with the number of dimensions. For realistic portfolio problems with 5–10 asset classes, multiple accounts, taxes, transaction costs, and health shocks, exact DP is completely infeasible.

The solution is **function approximation**: instead of storing $V(s)$ or $Q(s,a)$ for every possible (s,a) pair, we learn a parameterized function $Q(s,a;\mathbf{w})$ where $\mathbf{w}$ is a compact weight vector. The agent generalizes from states it has seen to states it hasn't.

This is the bridge from classical DP to reinforcement learning. The agent explores the state space by simulating lifetimes – thousands of them – and updates $\mathbf{w}$ based on observed transitions. It never needs to enumerate all states."

8 Slide 8 – Phase 3: RL Architecture

[Patrick speaks – ∼2 min]

"Let's talk about the design of Phase 3. We built a `Gym`-style environment where agents simulate 40-year lifetimes, making decisions at each year. The environment handles the stochastic transitions, reward computation, and terminal condition.

State featurization. The raw state (t, W, y, z) is mapped to an 8-dimensional feature vector:

- $t/T \in [0, 1]$: normalized time
- $\log(W)/\log(W_{\max}) \in [0, 1]$: log-normalized wealth (log scale because CRRA utility scales logarithmically with wealth)
- One-hot vector for income level: 3 dimensions
- One-hot vector for market regime: 3 dimensions

Total: 2 continuous + 3 + 3 one-hots = 8 features. Both continuous features are normalized to $[0, 1]$ to prevent gradient scaling issues.

Action space. In Phase 3: 11 consumption fractions $(0\%, 2\%, \ldots, 20\%)$ and 21 discrete portfolio allocations in 20% increments across 3 assets. Total: $11 \times 21 = 231$ discrete actions.

Four agents. We implemented two lightweight linear agents (Linear Q-learning and SARSA) and two neural network agents (DQN and PPO). The linear agents both use the same 12-feature expansion. The neural agents both use 128-unit two-layer networks.

A key distinction: Linear Q and DQN are *off-policy* – they learn from the optimal greedy action regardless of what they actually do during exploration. SARSA and PPO are *on-policy* – they learn about the policy they're actually executing. This distinction has both theoretical and empirical consequences, which we'll discuss.

Evaluation protocol. After training, all agents are evaluated on 2,000 Monte Carlo paths with $\epsilon = 0$ (pure greedy). Baselines are also evaluated on these same 2,000 paths for a fair comparison."

9 Slide 9 – Linear Q-Learning

[Patrick speaks – ∼2.5 min]

"Linear Q-learning is the simplest of our four agents, and in many ways the most instructive for understanding RL fundamentals.

Function approximation. We approximate the Q-function linearly:

$$Q(s, a; \mathbf{w}) = \mathbf{w}_a^\top \boldsymbol{\phi}(s),$$

where $\boldsymbol{\phi}(s) \in \mathbb{R}^{12}$ is the feature vector and $\mathbf{w}_a \in \mathbb{R}^{12}$ is a separate weight vector for each action. We maintain one weight vector per action, so the full parameter matrix $\mathbf{W}$ has shape $231 \times 12 = 2{,}772$ parameters total.

Feature engineering. The raw 8-dim featurization captures the state, but a purely linear function cannot represent interactions between variables – for instance, the relationship between time and wealth may be nonlinear. So we expand to 12 features by appending:

- t^2_{feat} (quadratic in time – captures the accelerating urgency as retirement approaches)
- w^2_{feat} (quadratic in wealth – captures diminishing marginal value)
- $t_{\text{feat}} \cdot w_{\text{feat}}$ (interaction term – captures how optimal allocation depends on *both* how old you are and how wealthy you are simultaneously)
- A constant bias term

So $\boldsymbol{\phi}(s) = [s_1, \ldots, s_8, t^2, w^2, t \cdot w, 1] \in \mathbb{R}^{12}$.

The update rule. At each time step, after observing transition (S, A, R, S'):

$$\mathbf{w}_A \leftarrow \mathbf{w}_A + \alpha \cdot \underbrace{\left(R + \gamma \max_{a'} Q(S', a'; \mathbf{w}) - Q(S, A; \mathbf{w})\right)}_{\delta_t\text{: TD error}} \cdot \boldsymbol{\phi}(S).$$

This is the *semi-gradient* update. We take the gradient of $Q(S, A; \mathbf{w})$ with respect to $\mathbf{w}_A$ – which equals $\boldsymbol{\phi}(S)$ for the linear model – but we treat the target $R + \gamma \max_{a'} Q(S', a')$ as a fixed constant. This is not a true gradient of any loss because the target also depends on $\mathbf{w}$; treating it as fixed is what makes it "semi-gradient." Treating both sides as dependent on $\mathbf{w}$ would give residual gradient methods, which are more stable but much slower to converge in practice.

Off-policy nature. The key phrase is $\max_{a'} Q(S', a'; \mathbf{w})$: we bootstrap from the *best possible* next action, regardless of what action the agent will actually take due to ϵ-greedy exploration. This makes Q-learning off-policy – it learns about the greedy policy while following an exploratory behavior policy.

Epsilon-greedy schedule. We use a GLIE (Greedy in the Limit with Infinite Exploration) schedule: ϵ decays linearly from 1.0 to 0.05 over 2,000 episodes, then stays at 0.05. Starting at $\epsilon = 1$ means the agent takes purely random actions initially. This is essential for exploration in the early phase – the agent needs to discover that zero-consumption actions are severely penalized before it can learn to avoid them.

The deadly triad. Combining (1) function approximation, (2) bootstrapping (using $Q(S')$ as the target), and (3) off-policy learning creates what Sutton & Barto call the *deadly triad*. In theory, this combination can cause Q-value divergence. In practice, it's stable here because the linear approximation is well-behaved (bounded features, small learning rate $\alpha = 10^{-4}$), and the problem has a finite horizon (no infinite-value spirals).

Result: $\mathbb{E}[\text{Utility}] = -1.59$, the second-best of our four agents."

10 Slide 10 – SARSA: On-Policy TD Control

[Patrick speaks – ∼3 min]

"SARSA shares the same architecture as Linear Q – identical feature expansion, identical 2,772 parameters, identical ϵ-greedy schedule. But there is one critical difference in the update rule, and that difference has profound theoretical and practical consequences.

The SARSA update rule. The name SARSA comes from the quintuple (S, A, R, S', A') used in each update:

$$\mathbf{w}_A \leftarrow \mathbf{w}_A + \alpha \cdot \left(R + \gamma\, Q(S', A'; \mathbf{w}) - Q(S, A; \mathbf{w})\right) \cdot \boldsymbol{\phi}(S).$$

Compare to Q-learning: where Q-learning uses $\max_{a'} Q(S', a')$, SARSA uses $Q(S', A')$ – where A' is the *actual next action* the agent will take under its ϵ-greedy policy. This is the on-policy distinction.

What does "on-policy" mean? Q-learning is *optimistic*: it assumes the future will be navigated greedily. SARSA is *honest*: it accounts for the fact that during training, the agent will sometimes take random exploratory actions. In a state where the greedy action is great but nearby actions are catastrophic, Q-learning will still bootstrap from the greedy value. SARSA will blend in the value of random exploration – making it more cautious.

In financial terms: SARSA calibrates its value estimates to a policy that includes the possibility of making suboptimal decisions. This yields a more conservative policy – but also a more realistic one, since no investor acts perfectly optimally in every situation.

The critical implementation detail: caching A'. A common implementation mistake is to select A' in the update, use it for bootstrapping, but then *re-sample* a fresh action for the next step. This silently converts SARSA into Q-learning (approximately), because the action you bootstrap from is not the action you actually execute.

Our implementation avoids this with an explicit caching mechanism:

1. In `update()`: we call `_epsilon_greedy_action(next_state)` to select A', use $Q(S', A')$ in the target, and *store* A' in `_cached_next_action`.
2. In `select_action()`: if the cache is set, we return the cached action (clearing it). Otherwise we sample fresh.

This guarantees that the A' used to compute the TD target is *identical* to the A' actually executed in the next environment step. Without this, SARSA's on-policy guarantee is broken.

Theoretical convergence. For *tabular* SARSA (no function approximation), convergence to Q^* is guaranteed under two conditions:

1. **GLIE** (Greedy in the Limit with Infinite Exploration): $\epsilon \to 0$ over time (satisfied by our linear decay), and every state-action pair is visited infinitely often.
2. **Robbins-Monro step sizes**: $\sum_t \alpha_t = \infty$ and $\sum_t \alpha_t^2 < \infty$. We use a constant $\alpha = 10^{-4}$, which violates the Robbins-Monro condition. This is a practical trade-off: constant step sizes converge to a neighborhood of Q^* rather than exactly to it, but they track non-stationarities better than diminishing rates.

With linear function approximation, convergence to exact Q^* is no longer guaranteed – but SARSA avoids the deadly triad (it's on-policy), making it the most theoretically grounded of our four agents.

Connection to textbook. This is Section 12.4 of Sutton & Barto, the linear semi-gradient SARSA algorithm. The update equation exactly matches Equation 12.5 in the textbook.

Result and interpretation. SARSA achieves $\mathbb{E}[\text{Utility}] = -1.88$ – more conservative than Linear Q's -1.59, but with the tightest downside risk. The 10th-percentile terminal wealth for SARSA is higher than for any other agent. This makes economic sense: on-policy training makes the agent aware that exploration will sometimes cause suboptimal outcomes, so it learns to protect against downside scenarios rather than optimizing only for the best case.

The gap between SARSA and Linear Q (-1.88 vs -1.59) is entirely attributable to the on-policy vs off-policy distinction – same features, same hyperparameters, different one line of code."

11 Slide 11 – DQN: Deep Q-Network

[Jack speaks – ∼2 min]

"DQN replaces the linear function approximator with a neural network. The key motivation: linear models with hand-crafted features are limited. Even with our quadratic expansion, the linear model cannot capture arbitrary nonlinear interactions between time, wealth, income, and regime. A deep network can.

Network architecture. DQN uses a feedforward Q-network with two hidden layers of 128 units each and ReLU activations. The input is the 8-dimensional state feature vector, the output is a 231-dimensional vector of Q-values – one per action. The agent selects the action with the highest Q-value (argmax). We use the Huber (SmoothL1) loss rather than MSE, which is less sensitive to large TD errors.

The two DQN innovations. Naive neural network Q-learning is unstable for two reasons: (1) sequential transitions are highly correlated, so gradient updates are biased toward recent experience; and (2) the TD target $R + \gamma \max_{a'} Q(S', a'; \mathbf{w})$ changes with every update to $\mathbf{w}$, creating a moving target problem. DQN addresses both:

- **Experience replay**: we maintain a circular buffer of 50,000 transitions. At each training step, we sample a uniformly random mini-batch of 64 transitions. This breaks temporal correlations and allows each transition to be replayed multiple times, improving sample efficiency.
- **Target network**: a separate, frozen copy of the Q-network $\hat{\mathbf{w}}$ that is only updated every 50 gradient steps (hard copy). The TD target is computed using this frozen network: $R + \gamma \max_{a'} Q(S', a'; \hat{\mathbf{w}})$. This stabilizes the target during gradient updates.

Gradient clipping. We clip gradients to norm 10 before each update. This prevents occasional large TD errors from causing catastrophic weight updates – especially important early in training when the network's Q-estimates are far from the true values.

Training logistics. DQN does not start learning until the replay buffer has at least 64 experiences (one batch). It trains on every step of every episode. This means significantly more computation per episode than the linear agents – DQN takes approximately 20 minutes to train versus 15–18 seconds for Linear Q or SARSA.

Result: $\mathbb{E}[\text{Utility}] = -1.54$, the best of all four agents. The neural network's flexibility to capture nonlinear state-action value relationships pays off."

12 Slide 12 – PPO: Proximal Policy Optimization

[Jack speaks – ∼2 min]

"PPO takes a fundamentally different approach. Instead of learning Q-values and deriving a policy implicitly as $\pi(s) =_a Q(s, a)$, PPO *directly* parameterizes the policy as a probability distribution over actions and optimizes it with policy gradient methods.

Actor-critic architecture. Our PPO uses a shared-trunk actor-critic: two 128-unit ReLU hidden layers shared between actor and critic, then separate output heads. The actor head outputs 231 logits – one per action – converted to a softmax probability distribution. The critic head outputs a scalar state value $V(s)$.

Policy gradient intuition. The policy gradient theorem says: update the policy parameters in the direction that increases the probability of actions that have positive *advantage* – meaning the action was better than average for that state. The advantage of taking action a in state s is $A(s,a) = Q(s,a) - V(s)$: how much better was this action than the typical action?

We estimate advantages using **Generalized Advantage Estimation (GAE)** with $\lambda = 0.95$:

$$\hat{A}_t = \sum_{k=0}^{T-t} (\gamma\lambda)^k \delta_{t+k}, \quad \delta_t = R_t + \gamma V(S_{t+1}) - V(S_t).$$

GAE interpolates between TD(0) (low variance, high bias, $\lambda = 0$) and Monte Carlo returns (high variance, zero bias, $\lambda = 1$). With $\lambda = 0.95$, we get low-bias estimates that are less noisy than full Monte Carlo.

The clipped surrogate objective. Vanilla policy gradient can make destructively large updates. PPO prevents this by clipping the probability ratio between the new and old policy:

$$\mathcal{L}^{\text{CLIP}} = \mathbb{E}\Big[\min\Big(r_t(\theta)\hat{A}_t,\ \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t\Big)\Big], \quad r_t = \frac{\pi_\theta(a|s)}{\pi_{\theta_{\text{old}}}(a|s)}.$$

With clip range $\epsilon = 0.2$, the policy cannot change by more than 20% per update. The full PPO loss adds a value function term (vf_coef = 0.5) and an entropy bonus (ent_coef = 0.01) to encourage exploration.

On-policy training. PPO collects 512 steps of experience (rollout buffer), then runs 4 epochs of mini-batch updates over that data (mini-batch size 64), then *discards* the buffer and collects fresh data. This is on-policy learning – unlike DQN, old data cannot be reused.

Result: $\mathbb{E}[\text{Utility}] = -1.95$. PPO has the most parameters (128-unit shared trunk with actor and critic heads) but does not achieve the best utility – the on-policy constraint and the policy gradient variance make it harder to optimize than off-policy value-based methods in this discrete-action setting."

13 Slide 13 – Training Convergence

[Jeffrey speaks – ∼1.5 min]

"The training convergence plots show the 100-episode running average of episode returns (total discounted utility collected in one simulated lifetime). Let me walk through what each curve tells us.

Early phase: all agents struggle. At $\epsilon = 1.0$, all agents take uniformly random actions. Many of these set consumption fraction to zero – which the CRRA utility function penalizes with a very large negative reward ($u(0) = -\infty$, approximated as a large finite penalty). So early returns are very negative.

Linear Q and SARSA converge quickly – within 2,000–4,000 episodes – and plateau around -2 to -2.5 in training return. Their learning curves are nearly identical in shape, which is expected: same architecture, same feature expansion, same ϵ schedule. The small separation – Linear Q slightly higher – reflects the off-policy vs on-policy distinction.

DQN is slowest to start improving because it requires filling a replay buffer before gradient updates begin, and neural network training is inherently noisier than linear regression. DQN also uses a

longer ϵ decay (3,000 episodes) to explore more thoroughly. But it ultimately converges to the best utility.

PPO often shows high training returns early because its entropy bonus encourages it to spread probability across many actions, avoiding the zero-consumption trap quickly. But its evaluation utility is lower than the value-based methods – training return is not a perfect proxy for evaluation performance.

Training times: Linear Q and SARSA finish 15,000 episodes in 15–18 seconds each. DQN takes approximately 20 minutes. PPO takes 2–3 minutes. For practical applications, the linear agents are the clear efficiency winners."

14 Slide 14 – Monte Carlo Evaluation

[Jeffrey speaks – ∼2 min]

"The Monte Carlo evaluation is our primary performance metric. We run 2,000 simulated 40-year lifetimes for each agent and each baseline, with $\epsilon = 0$ (agents act greedily, no exploration). We report expected total discounted CRRA utility.

The headline result: every RL agent outperforms every baseline. This is not a small margin – the gap between the best RL agent (DQN, -1.54) and the best baseline (60/40 balanced, -2.63) is enormous relative to the range of achievable utilities.

RL agent ranking:

Agent	$\mathbb{E}$[Utility]	Parameters
DQN	-1.54	∼75K
Linear Q	-1.59	2,772
SARSA	-1.88	2,772
PPO	-1.95	∼80K
60/40 Balanced	-2.63	–
100% Stocks	-2.71	–
100% Bonds	-2.75	–

The wealth paradox. RL agents accumulate far less terminal wealth than baselines – roughly \$90–115 at retirement versus \$490 for the 60/40 strategy. But they achieve much higher utility. How?

The resolution is the CRRA objective. With $\gamma = 2$, the agent values smooth consumption over a 40-year horizon far more than a large but volatile lump sum at retirement. Consider: consuming \$10/year for 40 years gives $\sum_{t=0}^{39} 0.96^t \cdot (-1/10) \approx -2.5$. Consuming \$1/year and ending with \$400 at retirement gives roughly -25 plus the terminal term. The math strongly favors steady consumption.

The baselines use a 4% annual withdrawal rule with fixed allocation. They mostly accumulate wealth rather than consuming it efficiently, which under CRRA utility is the wrong objective. The RL agents *discovered this from scratch* – they were only given the CRRA utility function as a reward signal, and they learned to smooth consumption over time.

This is the central result of the project: RL optimizes the actual objective. Rules of thumb that

target wealth implicitly optimize a different objective."

15 Slide 15 – Learned Policies: What Do the Agents Do?

[Jeffrey speaks – ∼2 min]

"The policy heatmaps give us interpretability: what allocation does each agent choose as a function of time (x-axis) and wealth (y-axis)? Color represents stock allocation, from 0% (blue) to 100% (red). All four agents are shown side by side.

The glidepath pattern. Despite having radically different architectures, all four agents independently discover the same core economic insight: invest heavily in equities when young, gradually shift toward bonds and cash as retirement approaches. This is the glidepath that classical Merton theory predicts is optimal. The agents discovered it without being given any economic theory – only the reward signal.

Wealth-dependent allocation. Unlike a fixed glidepath, all four agents also condition on wealth. At low wealth levels (bottom of each heatmap), agents tilt more toward stocks – the marginal expected gain from equity risk is worth more when you have little to lose and much to gain. At high wealth, agents are more conservative – you don't need to take equity risk if you already have enough.

Differences in style.

- **Linear Q and SARSA**: similar overall pattern, but sharper discontinuities – the limited capacity of a linear model creates cleaner decision boundaries between action regions. SARSA's policy is slightly more conservative (lower stock allocations overall) reflecting its on-policy training.
- **DQN**: smoother policy surface because the neural network naturally interpolates between nearby states. The network can represent gradual transitions rather than discrete jumps.
- **PPO**: tends toward higher stock allocations on average, which is consistent with the entropy bonus encouraging broader action distribution – PPO doesn't get stuck in low-stock modes as easily.

Regime conditioning. All agents allocate more to stocks in bull regimes and more to bonds in bear regimes. This regime-switching behavior is optimal – the expected return on equities is much higher in bull conditions – and all agents learn it.

The key takeaway: interpretable heatmaps confirm that our RL agents are not black boxes. They learned economically sensible policies."

16 Slide 16 – Wealth Trajectories and Terminal Wealth

[Jeffrey speaks – ∼1 min]

"These plots tell the story of what the policies actually *do* over a lifetime.

Left panel – median wealth trajectories. The baselines grow monotonically: they consume little and invest aggressively, so wealth compounds. The 100% stocks baseline reaches median terminal wealth of over $500 by period 40. RL agents show a different pattern: wealth rises initially as income and returns compound, but then flattens or slightly declines as the agent increases

consumption. Median terminal wealth for RL agents is \$90–115.

Right panel – terminal wealth distributions. Baselines have wide, right-skewed distributions with fat tails – they benefit enormously from bull markets but are also exposed to bear markets. RL agents have tighter, more concentrated distributions. They sacrificed the upside lottery in favor of consumption during the journey.

The economic interpretation. Under $\gamma = 2$ CRRA utility, the value of an extra dollar of consumption is $u'(c) = 1/c^2$. At $c = 10$ (typical mid-life RL agent consumption), $u'(10) = 0.01$. At $c = 100$ (terminal wealth liquidated), $u'(100) = 0.0001$. Consuming now is 100 times more valuable per dollar than leaving it for the terminal period, because of diminishing marginal utility.

The agents internalized this convexity. They traded terminal wealth – worth very little in utility terms – for smooth lifetime consumption."

17 Slide 17 – Comparing All Four RL Agents

[Jack speaks – ∼1.5 min]

"Let's compare the four agents directly across the dimensions that matter.

On expected utility: DQN wins, then Linear Q, then SARSA, then PPO. The off-policy methods (DQN and Linear Q) both outperform the on-policy methods (SARSA and PPO). This is consistent with theory: off-policy bootstrapping from the greedy action is more aggressive and achieves higher utility at the cost of being slightly overoptimistic.

On downside protection: SARSA has the best 10th-percentile outcomes. Its on-policy update makes it aware of exploration costs and more conservative in bad states. For a risk-averse investor, this might be the most important metric.

On compute: the gap is enormous. Linear Q and SARSA train in 15–18 seconds for 15,000 episodes with 2,772 parameters. DQN takes ∼20 minutes with 75K parameters. The utility difference between Linear Q (-1.59) and DQN (-1.54) is 3% – a tiny performance gap for a 60× increase in training time and 27× more parameters.

The practical recommendation: for production deployment in a lifecycle allocation system, linear agents are the pragmatic choice. They are fast to train, easy to retrain as market conditions change, interpretable (you can inspect the 2,772 weights and understand what they represent), and perform nearly as well as the neural network agents.

The broader lesson: architecture complexity is not the dominant factor. The most important decision was *what objective to optimize* – and all four agents, once pointed at the right objective (CRRA utility), converged on economically sensible policies."

18 Slide 18 – Conclusions

[Patrick speaks – ∼1.5 min]

"Let me close with the key takeaways from this project.

1. Exact DP works but doesn't scale. Backward induction gives the provably optimal solution for a small, discretized problem in milliseconds. But it fails completely as we add income states, regimes, asset classes, or continuous state variables. This motivates RL.

2. RL handles the richer problem. All four of our RL agents – ranging from 2,772-parameter linear models to 80K-parameter neural networks – learn sensible policies in the 40-year, 3-income, 3-regime Phase 3 problem. They generalize across wealth levels without explicitly discretizing.

3. All RL agents beat all baselines. The utility improvement is massive – roughly 40% better than the best static baseline. This is primarily because RL optimizes the actual objective (CRRA utility) while baselines implicitly optimize wealth accumulation.

4. The off/on-policy distinction matters empirically. Off-policy methods (Linear Q, DQN) achieve higher expected utility. On-policy methods (SARSA, PPO) are more conservative and have better downside protection. The right choice depends on the investor's priorities.

5. Simple models are competitive. Linear Q with 2,772 parameters achieves 97% of DQN's utility. For practical deployment, simplicity, speed, and interpretability strongly favor the linear agents.

Future directions: continuous action spaces via DDPG or SAC would eliminate the discrete action grid; adding transaction costs and taxes would make the model more realistic; calibrating parameters to real historical data would make it usable for actual financial planning."

19 Slide 19 – Thank You & Q&A

[All – open for questions]

Anticipated Questions

Q: Why does RL have lower terminal wealth but higher utility?

Because CRRA utility with $\gamma = 2$ is $u(c) = -1/c$. The marginal utility of consumption is $u'(c) = 1/c^2$, which falls rapidly. Consuming \$10 generates 100 times more marginal utility per dollar than consuming \$100. So smooth consumption across 40 years is worth far more in aggregate than a large lump sum at retirement. The baselines accumulate wealth efficiently but use a 4% withdrawal rule that is calibrated for wealth sustainability, not CRRA utility maximization.

Q: Why is $\gamma_{\mathbf{RL}} = 1.0$ in the agents if $\beta = 0.96$?

Because the discount factor is already incorporated into each reward: we compute $R_t = \beta^t \cdot u(C_t)$ before passing it to the agent. If we also set $\gamma_{\text{RL}} = 0.96$ in the TD update, we would double-discount – applying β^t once in the reward and once more in the Bellman equation. Setting $\gamma_{\text{RL}} = 1$ is correct; the agent receives rewards that already reflect the time value of consumption.

Q: Why do off-policy methods outperform on-policy methods here?

Off-policy methods (Linear Q, DQN) bootstrap from $\max_{a'} Q(S', a')$ – the value under the optimal greedy policy – regardless of whether the agent will actually execute that action. On-policy methods (SARSA, PPO) bootstrap from the value of the action they will actually execute, including random exploration. In a problem with strong equity premiums and a clear directional objective (maximize lifetime utility), the aggressive off-policy updates learn a better policy faster. If the environment were more adversarial (e.g., if exploratory actions caused catastrophic, irreversible losses), on-policy methods would be preferred because they account for the risk of exploration.

Q: Is SARSA convergence guaranteed with function approximation?

No. The tabular SARSA convergence theorem (convergence to Q^* under GLIE and Robbins-Monro step sizes) does not extend to function approximation in general. However, on-policy linear function approximation avoids the deadly triad (function approximation + bootstrapping + off-policy), making it the most theoretically safe configuration. With linear FA, the update is well-conditioned and the eigenvalues of the update operator are typically bounded, giving empirical stability even without a formal convergence guarantee.

Q: Why not use SARSA(λ) with eligibility traces?

Eligibility traces (SARSA(λ)) propagate credit backwards through recent states simultaneously, which can speed up learning in environments with long credit assignment chains. With 40-period episodes and relatively short credit assignment distances, TD(0) already propagates value reasonably well. Adding λ introduces a hyperparameter and additional memory (one eligibility trace per feature per action). It's a natural extension for future work.

Q: How close are the RL agents to the true optimum?

We ran the Phase 2 backward-induction DP solver on a discretized version of the Phase 3 environment. The DP achieves $\mathbb{E}[\text{Utility}] \approx -1.46$ on the discretized grid. DQN at -1.54 has approximately a 5% optimality gap. Linear Q at -1.59 has about an 9% gap. Given that the DP uses a much coarser discretization of wealth and is solved on the exact same grid (not a generalization), the RL agents are remarkably close to optimal.

Q: What is the deadly triad and which agents avoid it?

The deadly triad (Sutton & Barto, Chapter 11) is the combination of: (1) function approximation, (2) bootstrapping (using an estimated value as the TD target), and (3) off-policy learning. All three together can cause the estimated Q-values to diverge to infinity. SARSA avoids it by being on-policy. DQN works around it with a target network (reduces the severity of bootstrapping a moving target) and experience replay (reduces correlation). Linear Q faces the full triad but in practice remains stable because the linear model is not expressive enough to enter a divergence spiral.

Q: Why use a discrete action space rather than continuous?

Discrete actions make it straightforward to implement Q-learning and SARSA with linear function approximation – the Q-values are indexed by action and we can take efficiently. Continuous action spaces require different algorithms (DDPG, SAC, TD3) that parameterize the policy as a neural network. The 231-action discrete space provides sufficient resolution for the consumption-allocation problem. A finer grid would improve solution quality but increase computation linearly. Continuous action methods are a natural extension.